Module 1

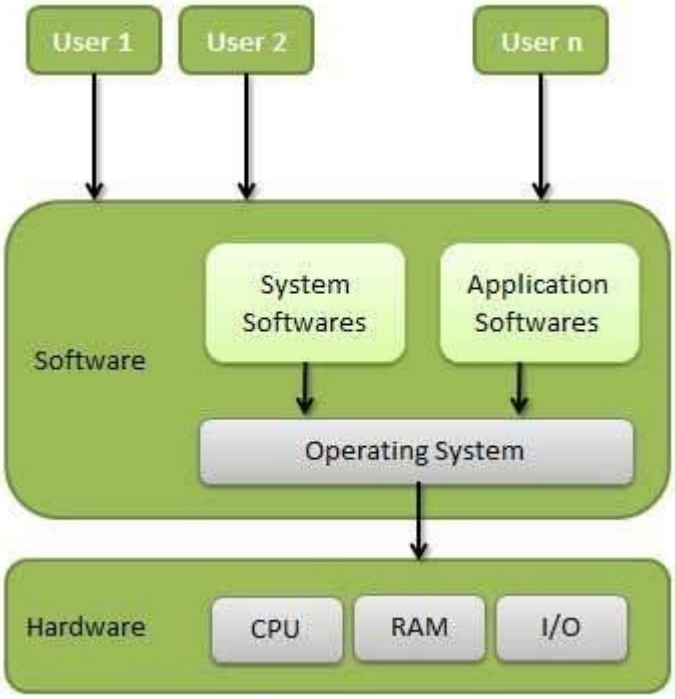
Operating system overview

An Operating System (OS) is an interface between a computer user and computer hardware. An operating system is a software which performs all the basic tasks like file management, memory management, process management, handling input and output, and controlling peripheral devices such as disk drives and printers.

Some popular Operating Systems include Linux Operating System, Windows Operating System, VMS, OS/400, AIX, z/OS, etc.

Definition

An operating system is a program that acts as an interface between the user and the computer hardware and controls the execution of all kinds of programs.



Following are some of important functions of an operating System.

- Memory Management
- Processor Management
- Device Management
- File Management
- Security
- Control over system performance
- Job accounting
- Error detecting aids

- Coordination between other software and users

Memory Management

Memory management refers to management of Primary Memory or Main Memory. Main memory is a large array of words or bytes where each word or byte has its own address.

Main memory provides a fast storage that can be accessed directly by the CPU. For a program to be executed, it must in the main memory. An Operating System does the following activities for memory management −

- Keeps tracks of primary memory, i.e., what part of it are in use by whom, what part are not in use.
- In multiprogramming, the OS decides which process will get memory when and how much.
- Allocates the memory when a process requests it to do so.
- De-allocates the memory when a process no longer needs it or has been terminated.

Processor Management

In multiprogramming environment, the OS decides which process gets the processor when and for how much time. This function is called **process scheduling**. An Operating System does the following activities for processor management −

- Keeps tracks of processor and status of process. The program responsible for this task is known as **traffic controller**.
- Allocates the processor (CPU) to a process.
- De-allocates processor when a process is no longer required.

Device Management

An Operating System manages device communication via their respective drivers. It does the following activities for device management −

- Keeps tracks of all devices. Program responsible for this task is known as the **I/O controller**.
- Decides which process gets the device when and for how much time.
- Allocates the device in the efficient way.
- De-allocates devices.

File Management

A file system is normally organized into directories for easy navigation and usage. These directories may contain files and other directions.

An Operating System does the following activities for file management −

- Keeps track of information, location, uses, status etc. The collective facilities are often known as **file system**.
- Decides who gets the resources.
- Allocates the resources.
- De-allocates the resources.

Other Important Activities

Following are some of the important activities that an Operating System performs −

- **Security** − By means of password and similar other techniques, it prevents unauthorized access to programs and data.
- **Control over system performance** − Recording delays between request for a service and response from the system.
- **Job accounting** − Keeping track of time and resources used by various jobs and users.
- **Error detecting aids** − Production of dumps, traces, error messages, and other debugging and error detecting aids.
- **Coordination between other softwares and users** − Coordination and assignment of compilers, interpreters, assemblers and other software to the various users of the computer systems.

Objectives of Operating System

The objectives of the operating system are −

- To make the computer system convenient to use in an efficient manner.
- To hide the details of the hardware resources from the users.
- To provide users a convenient interface to use the computer system.
- To act as an intermediary between the hardware and its users, making it easier for the users to access and use other resources.
- To manage the resources of a computer system.
- To keep track of who is using which resource, granting resource requests, and mediating conflicting requests from different programs and users.
- To provide efficient and fair sharing of resources among users and programs.

Evolution of Operating Systems:

1. Serial processing

2. Batch processing

3. Multiprogramming

4. Multitasking or time sharing System

1. Serial Processing:

● Early computer from late 1940 to the mid 1950.

● The programmer interacted directly with the computer hardware.

● These machine are called bare machine as they don't have OS.

● Every computer system is programmed in its machine language.

● Uses Punch Card, paper tapes and language translator These system presented two major problems. 1. Scheduling 2. Set up time: Scheduling: Used sign up sheet to reserve machine time. A user may sign up for an hour but finishes his job in 45 minutes. This would result in wasted computer idle time, also the user might run into the problem not finish his job in alloted time. Set up time: A single program involves:

● Loading compiler and source program in memory

● Saving the compiled program (object code)

● Loading and linking together object program and common function Each of these steps involves the mounting or dismounting tapes on setting up punch cards. If an error occur user had to go the beginning of the set up sequence. Thus, a considerable amount of time is spent in setting up the program to run. This mode of operation is turned as serial processing ,reflecting the fact that users access the computer in series.

2. Simple Batch Processing:

● Early computers were very expensive, and therefore it was important to maximize processor utilization.

● The wasted time due to scheduling and setup time in Serial Processing was unacceptable.

● To improve utilization, the concept of a batch operating system was developed.

● Batch is defined as a group of jobs with similar needs. The operating system allows users to form batches. Computer executes each batch sequentially, processing all jobs of a batch considering them as a single process called batch processing. The central idea behind the simple batch-processing scheme is the use of a piece of software known as the monitor. With this type of OS, the user no longer has direct access to the processor. Instead, the user submits the job on cards or tape to a computer operator, who batches the jobs together sequentially and places the entire batch on an input device, for use by the monitor. Each program is constructed to branch back to the monitor when it completes processing, at which point the monitor automatically begins loading the next program.

With a batch operating system, processor time alternates between execution of user programs and execution of the monitor. There have been two sacrifices: Some main memory is now given over to the monitor and some processor time is consumed by the monitor. Both of these are forms of overhead.

3. Multiprogrammed Batch System:

A single program cannot keep either CPU or I/O devices busy at all times. Multiprogramming increases CPU utilization by organizing jobs in such a manner that CPU has always one job to execute.

If computer is required to run several programs at the same time, the processor could be kept busy for the most of the time by switching its attention from one program to the next. Additionally I/O transfer could overlap the processor activity i.e, while one program is awaiting for an I/O transfer, another program can use the processor.

So CPU never sits idle or if comes in idle state then after a very small time it is again busy.

4. Multitasking or Time Sharing System:

● Multiprogramming didn't provide the user interaction with the computer system.

● Time sharing or Multitasking is a logical extension of Multiprogramming that provides user interaction.

● There are more than one user interacting the system at the same time

● The switching of CPU between two users is so fast that it gives the impression to user that he is only working on the system but actually it is shared among different users.

● CPU bound is divided into different time slots depending upon the number of users using the system.

● just as multiprogramming allows the processor to handle multiple batch jobs at a time, multiprogramming can also be used to handle multiple interactive jobs. In this latter case, the technique is referred to as time sharing, because processor time is shared among multiple users

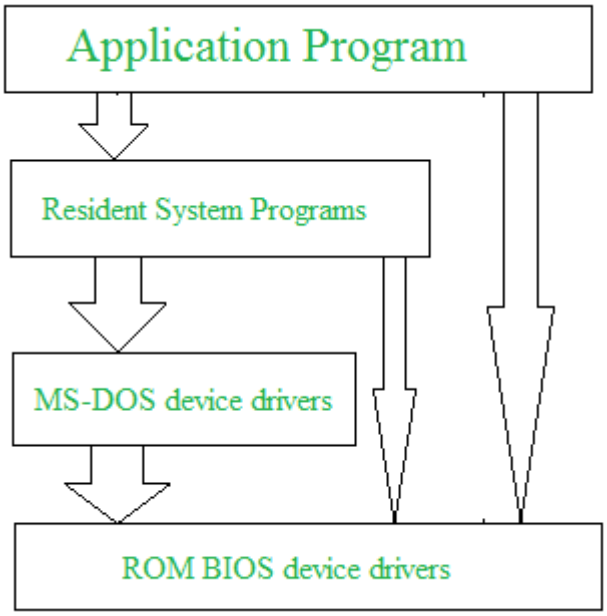
● A multitasking system uses CPU scheduling and multiprogramming to provide each user with a small portion of a time shared computer. Each user has at least one separate program in memory.

● Multitasking are more complex than multiprogramming and must provide a mechanism for jobs synchronization and communication and it may ensure that system does not go in deadlock. Although batch processing is still in use but most of the system today available uses the concept of multitasking and Multiprogramming.

Operating System Structure

Operating system can be implemented with the help of various structures. The structure of the OS depends mainly on how the various common components of the operating system are interconnected and melded into the kernel. Depending on this we have following structures of the operating system:

Simple structure:
Such operating systems do not have well defined structure and are small, simple and limited systems. The interfaces and levels of functionality are not well separated. MS-DOS is an example of such operating system. In MS-DOS application programs are able to access the basic I/O routines. These types of operating system cause the entire system to crash if one of the user programs fails.
Diagram of the structure of MS-DOS is shown below.



The operating system is split into various layers In the layered operating system and each of the layers have different functionalities. This type of operating system was created as an improvement over the early monolithic systems.

1. Layered Structure:-

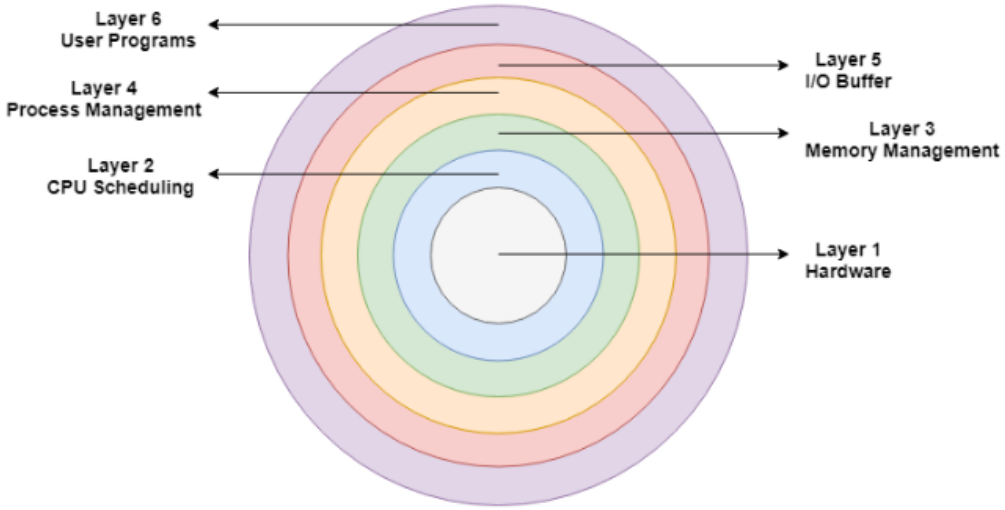
Why Layering in Operating System?

Layering provides a distinct advantage in an operating system. All the layers can be defined separately and interact with each other as required. Also, it is easier to create, maintain and update the system if it is done in the form of layers. Change in one layer specification does not affect the rest of the layers.

Each of the layers in the operating system can only interact with the layers that are above and below it. The lowest layer handles the hardware and the uppermost layer deals with the user applications.

Layers in Layered Operating System

There are six layers in the layered operating system. A diagram demonstrating these layers is as follows:



LAYERED OPERATING SYSTEM

Details about the six layers are:

Hardware

This layer interacts with the system hardware and coordinates with all the peripheral devices used such as printer, mouse, keyboard, scanner etc. The hardware layer is the lowest layer in the layered operating system architecture.

CPU Scheduling

This layer deals with scheduling the processes for the CPU. There are many scheduling queues that are used to handle processes. When the processes enter the system, they are put into the job queue. The processes that are ready to execute in the main memory are kept in the ready queue.

Memory Management

Memory management deals with memory and the moving of processes from disk to primary memory for execution and back again. This is handled by the third layer of the operating system.

Process Management

This layer is responsible for managing the processes i.e assigning the processor to a process at a time. This is known as process scheduling. The different algorithms used for process scheduling are FCFS (first come first served), SJF (shortest job first), priority scheduling, round-robin scheduling etc.

I/O Buffer

I/O devices are very important in the computer systems. They provide users with the means of interacting with the system. This layer handles the buffers for the I/O devices and makes sure that they work correctly.

User Programs

This is the highest layer in the layered operating system. This layer deals with the many user programs and applications that run in an operating system such as word processors, games, browsers etc.

Advantages :
There are several advantages to this design :

1. **Modularity :**
   This design promotes modularity as each layer performs only the tasks it is scheduled to perform.
2. **Easy debugging :**
   As the layers are discrete so it is very easy to debug. Suppose an error occurs in the CPU scheduling layer, so the developer can only search that particular layer to debug, unlike the Monolithic system in which all the services are present together.
3. **Easy update :**
   A modification made in a particular layer will not affect the other layers.
4. **No direct access to hardware :**
   The hardware layer is the innermost layer present in the design. So a user can use the services of hardware but cannot directly modify or access it, unlike the Simple system in which the user had direct access to the hardware.
5. **Abstraction :**
   Every layer is concerned with its own functions. So the functions and implementations of the other layers are abstract to it.

Disadvantages :
Though this system has several advantages over the Monolithic and Simple design, there are also some disadvantages as follows.

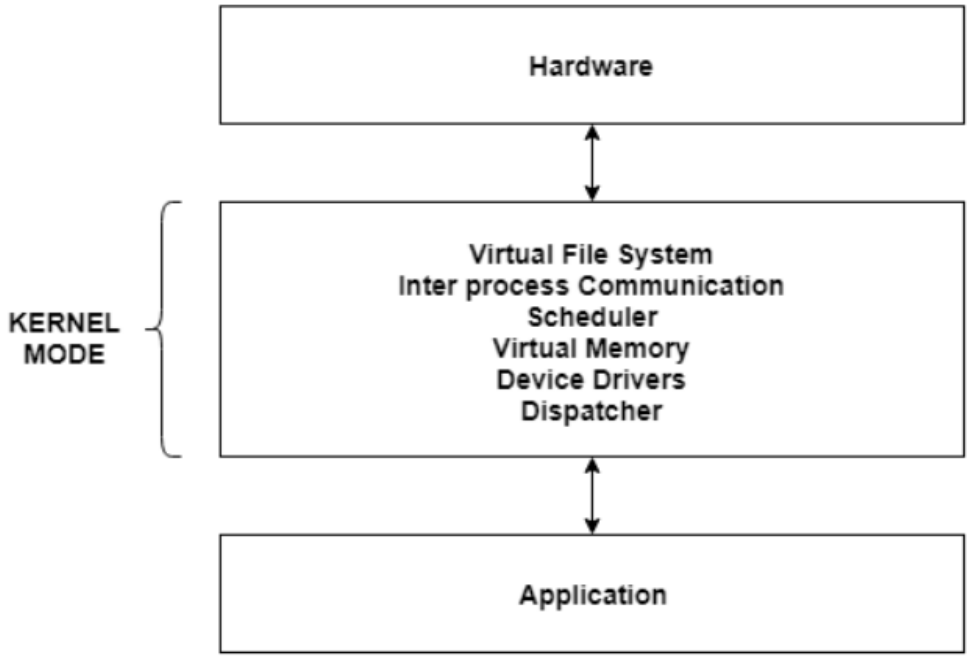
1. **Complex and careful implementation :**
   As a layer can access the services of the layers below it, so the arrangement of the layers must be done carefully. For example, the backing storage layer uses the services of the memory management layer. So it must be kept below the memory management layer. Thus with great modularity comes complex implementation.
2. **Slower in execution :**
   If a layer wants to interact with another layer, it sends a request that has to travel through all the layers present in between the two interacting layers. Thus it increases response time, unlike the Monolithic system which is faster than this. Thus an increase in the number of layers may lead to a very inefficient design.

Monolithic Kernel

- Earlier in this type of kernel architecture, all the basic system services like process and memory management, interrupt handling etc were packaged into a single module in kernel space.
- This type of architecture led to some serious drawbacks like 1) Size of kernel, which was huge. 2)Poor maintainability, which means bug fixing or addition of new features resulted in recompilation of the whole kernel which could consume hours
- In a modern day approach to monolithic architecture, the kernel consists of different modules which can be dynamically loaded and un-loaded.
- This modular approach allows easy extension of OS's capabilities. With this approach, maintainability of kernel became very easy as only the concerned module needs to be loaded and unloaded every time there is a change or bug fix in a particular module.
- So, there is no need to bring down and recompile the whole kernel for a smallest bit of change.

The entire operating system works in the kernel space in the monolithic system. This increases the size of the kernel as well as the operating system. This is different than the microkernel system where the minimum software that is required to correctly implement an operating system is kept in the kernel.

A diagram that demonstrates the architecture of a monolithic system is as follows −



Monolithic Kernel Operating System

The kernel provides various services such as memory management, file management, process scheduling etc. using function calls. This makes the execution of the operating system quite fast as the services are implemented under the same address space.

Advantages of Monolithic Kernel

Some of the advantages of monolithic kernel are −

- The execution of the monolithic kernel is quite fast as the services such as memory management, file management, process scheduling etc.are implemented under the same address space.
- A process runs completely in a single address space in the monolithic kernel.
- The monolithic kernel is a static single binary file.

Disadvantages of Monolithic Kernel

Some of the disadvantages of monolithic kernel are −

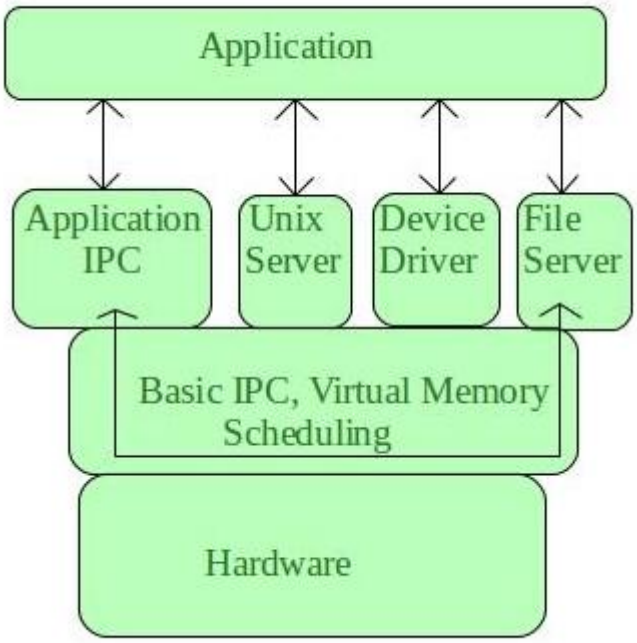
- If any service fails in the monolithic kernel, it leads to the failure of the entire system.
- To add any new service, the entire operating system needs to be modified by the user.

Microkernels

- This architecture majorly caters to the problem of ever growing size of kernel code which we could not control in the monolithic approach.
- This architecture allows some basic services like device driver management, protocol stack, file system etc to run in user space.
- This reduces the kernel code size and also increases the security and stability of OS as we have the bare minimum code running in kernel.
- So, if suppose a basic service like network service crashes due to buffer overflow, then only the networking service's memory would be corrupted, leaving the rest of the system still functional.
- In this architecture, all the basic OS services which are made part of user space are made to run as servers which are used by other programs in the system through inter process communication (IPC). eg: we have servers for device drivers, network protocol stacks, file systems, graphics, etc.
- Microkernel servers are essentially daemon programs like any others, except that the kernel grants some of them privileges to interact with parts of physical memory that are otherwise off limits to most programs. This allows some servers, particularly device drivers, to interact directly with hardware. These servers are started at the system start-up.

What is Microkernel?

Microkernel is one of the classification of the kernel. Being a kernel it manages all system resources. But in a microkernel, the **user services** and **kernel services** are implemented in different address space. The user services are kept in **user address space**, and kernel services are kept under **kernel address space**, thus also reduces the size of kernel and size of operating system as well.



- It provides minimal services of process and memory management.
- The communication between client program/application and services running in user address space is established through message passing, reducing the speed of execution microkernel.
- The Operating System **remains unaffected** as user services and kernel services are isolated so if any user service fails it does not affect kernel service. Thus it adds to one of the advantages in a microkernel.
- It is easily **extendable** i.e. if any new services are to be added they are added to user address space and hence requires no modification in kernel space.
- It is also portable, secure and reliable.

Microkernel Architecture –

Since kernel is the core part of the operating system, so it is meant for handling the most important services only. Thus in this architecture only the most important services are inside kernel and rest of the OS services are present inside system application program. Thus users are able to interact with those not-so important services within the system application. And the microkernel is solely responsible for the most important services of operating system they are named as follows:

- Inter process-Communication
- Memory Management
- CPU-Scheduling

Advantages of Microkernel –

- The architecture of this kernel is small and isolated hence it can function better.

- Expansion of the system is easier, it is simply added in the system application without disturbing the kernel.

Disadvantage of Microkernel

Here, are drawback/cons of using Microkernel:

- Providing services in a microkernel system are expensive compared to the normal monolithic system.
- Context switch or a function call needed when the drivers are implemented as procedures or processes, respectively.
- The performance of a microkernel system can be indifferent and may lead to some problems.

Differences Between Microkernel and Monolithic Kernel

Some of the differences between microkernel and monolithic kernel are given as follows −

- The microkernel is much smaller in size as compared to the monolithic kernel.
- The microkernel is easily extensible whereas this is quite complicated for the monolithic kernel.
- The execution of the microkernel is slower as compared to the monolithic kernel.
- Much more code is required to write a microkernel than the monolithic kernel.
- Examples of Microkernel are QNX, Symbian, L4 Linux etc. Monolithic Kernel examples areLinux, BSD etc.

Difference Between Microkernel and Monolithic Kernel

Parameters	Monolithic kernel	MicroKernel
Basic	It is a large process running in a single address space	It can be broken down into separate processes called servers.
Code	In order to write a monolithic kernel, less code is required.	In order to write a microkernel, more code is required

Security	If a service crashes, the whole system collapses in a monolithic kernel.	If a service crashes, it never affects the working of a microkernel.
Communication	It is a single static binary file	Servers communicate through IPC.
Example	Linux, BSDs, Microsoft Windows (95,98, Me), Solaris, OS-9, AIX, DOS, XTS-400, etc.	L4Linux, QNX, SymbianK42, Mac OS X, Integrity, etc.

Linux Kernel

Linux kernel is a free, open-source, monolithic, modular, Unix-like operating system kernel. It is the main component of the Linux operating system (OS) and is the core interface between the computer's hardware and its processes.

The Linux kernel is used by Linux distributions alongside GNU tools and libraries. This combination is sometimes referred to as GNU/Linux. Popular Linux distributions include Ubuntu, Fedora, and Arch Linux.

The Linux kernel was created by Linus Torvalds and is currently an open-source project with thousands of developers actively working on it.

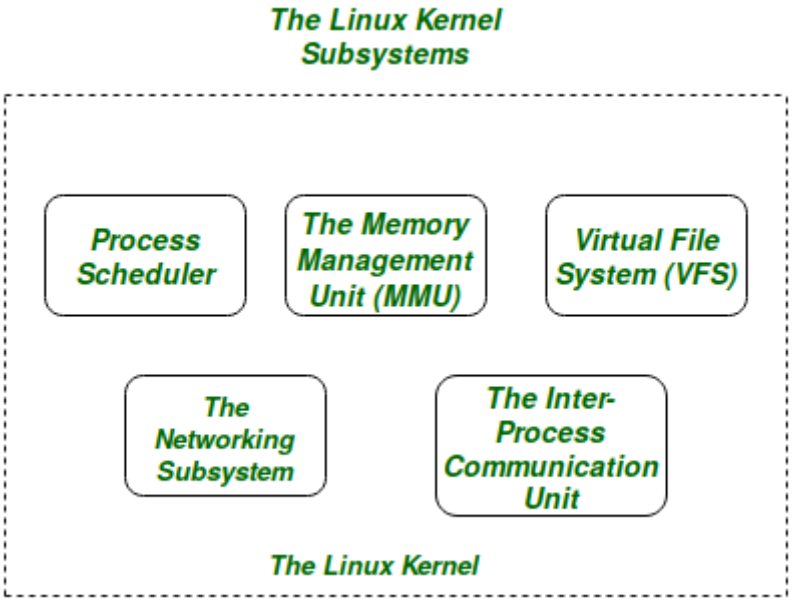


Figure: The Linux Kernel

The **Core Subsystems** of the **Linux Kernel** are as follows:

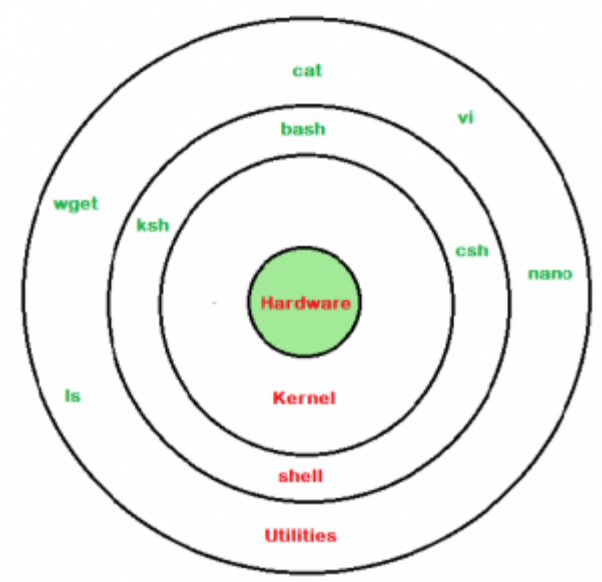
1. The Process Scheduler
2. The Memory Management Unit (MMU)
3. The Virtual File System (VFS)
4. The Networking Unit
5. Inter-Process Communication Unit

The basic functioning of each of the 1st three subsystems is elaborated below:

- **The Process Scheduler:**
  This kernel subsystem is responsible for fairly distributing the CPU time among all the processes running on the system simultaneously.
- **The Memory Management Unit:**
  This kernel sub-unit is responsible for proper distribution of the memory resources among the various processes running on the system. The MMU does more than just simply provide separate virtual address spaces for each of the processes.
- **The Virtual File System:**
  This subsystem is responsible for providing a unified interface to access stored data across different filesystems and physical storage media.

Shell

A shell is special user program which provide an interface to user to use operating system services. Shell accept human readable commands from user and convert them into something which kernel can understand. It is a command language interpreter that execute commands read from input devices such as keyboards or from files. The shell gets started when the user logs in or start the terminal.

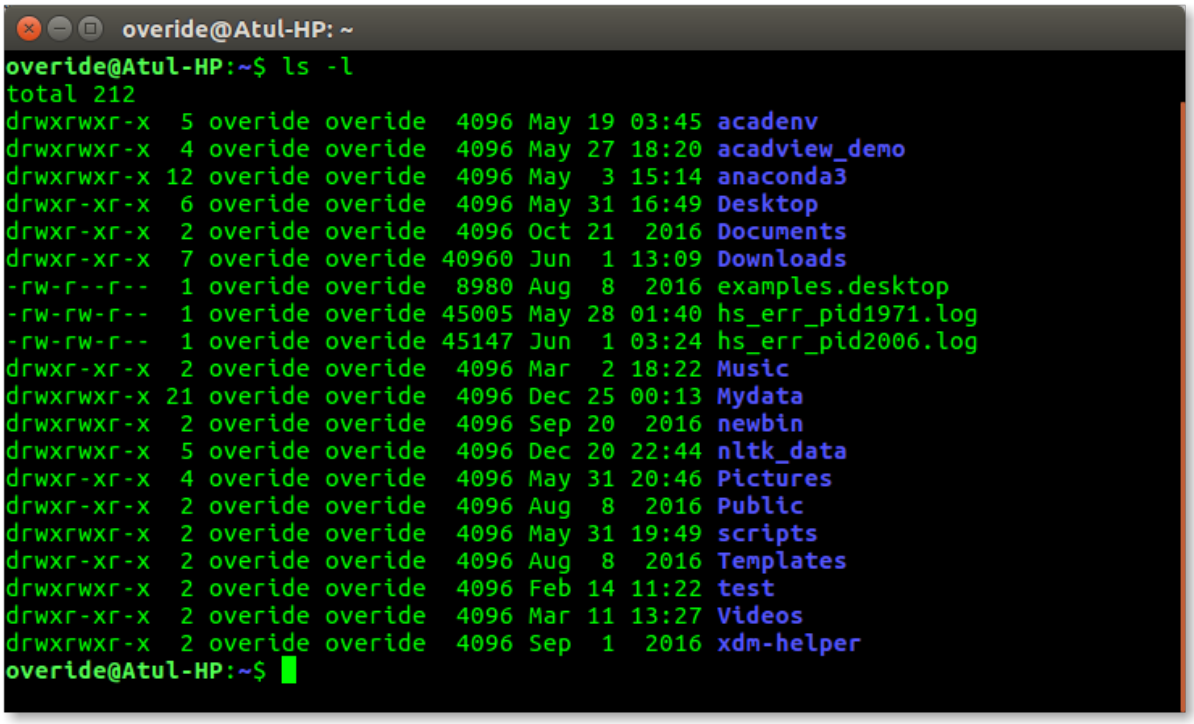


linux shell

- **Definition:** A shell is a program that acts as the interface between user and UNIX system, allowing user to enter commands for the operating system to execute.
- It is a command interpreter and programming language.
- It may be used interactively mode or non-interactively mode.
- It allows execution of synchronous commands and asynchronous commands.
- Shell is broadly classified into two categories –
    - Command Line Shell
    - Graphical shell

Command Line Shell

Shell can be accessed by user using a command line interface. A special program called **Terminal** in linux/macOS or **Command Prompt** in Windows OS is provided to type in the human readable commands such as “cat”, “ls” etc. and then it is being execute. The result is then displayed on the terminal to the user. A terminal in Ubuntu 16.4 system looks like this –

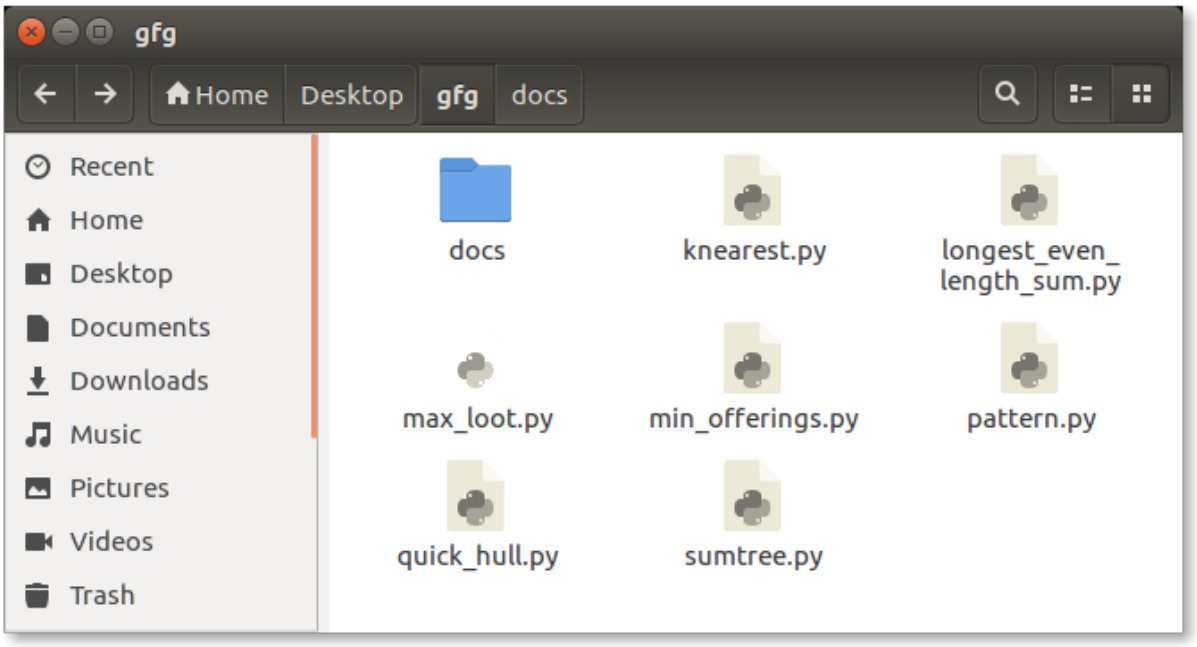


linux command line

In above screenshot **“ls”** command with **“-l”** option is executed.
It will list all the files in current working directory in long listing format.
Working with command line shell is bit difficult for the beginners because it’s hard to memorize so many commands. It is very powerful, it allows user to store commands in a file and execute them together. This way any repetitive task can be easily automated. These files are usually called **batch files** in Windows and **Shell** Scripts in Linux/macOS systems.

Graphical Shells

Graphical shells provide means for manipulating programs based on graphical user interface (GUI), by allowing for operations such as opening, closing, moving and resizing windows, as well as switching focus between windows. Window OS or Ubuntu OS can be considered as good example which provide GUI to user for interacting with program. User do not need to type in command for every actions.A typical GUI in Ubuntu system –



GUI shell

There are several shells are available for Linux systems like –

- BASH (Bourne Again SHell) – It is most widely used shell in Linux systems. It is used as default login shell in Linux systems and in macOS. It can also be installed on Windows OS.
- CSH (C SHell) – The C shell's syntax and usage are very similar to the C programming language.
- KSH (Korn SHell) – The Korn Shell also was the base for the POSIX Shell standard specifications etc.

Each shell does the same job but understand different commands and provide different built in functions.

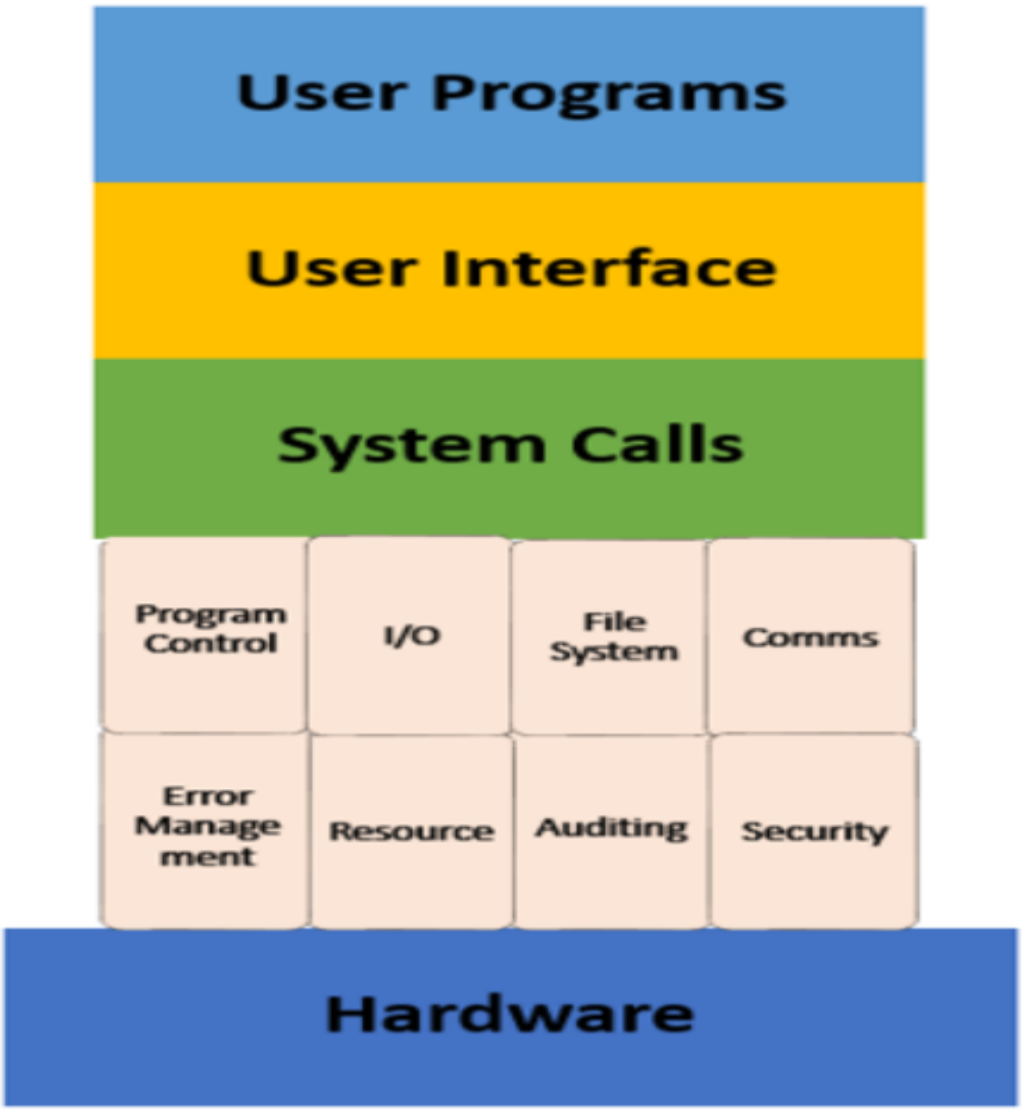
Shell Responsibilities

- Program execution
- Variable and filename substitution
- I/O Redirection
- Pipeline hookup
- Environment control
- Interpreted programming language

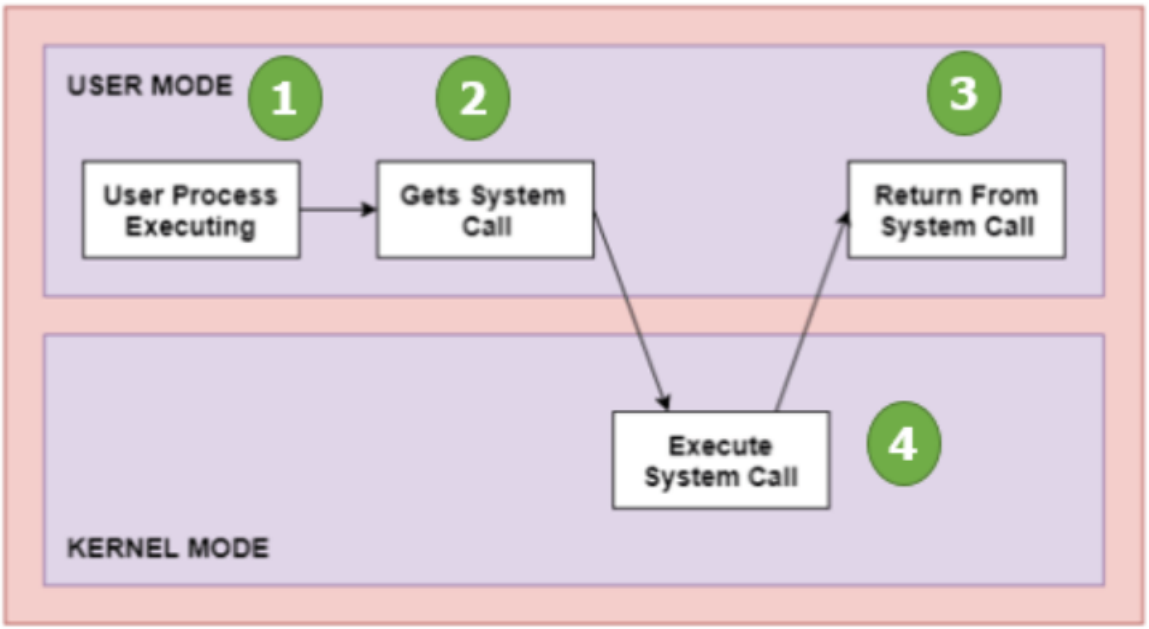
System Call

- **Definition:**

A system call is a mechanism that provides the interface between a process and the operating system. It is a programmatic method in which a computer program requests a service from the kernel of the OS.



How System Call Works?



As you can see in the above-given diagram.

Step 1) The processes executed in the user mode till the time a system call interrupts it.

Step 2) After that, the system call is executed in the kernel-mode on a priority basis.

Step 3) Once system call execution is over, control returns to the user mode.,

Step 4) The execution of user processes resumed in Kernel mode.

Why do you need System Calls in OS?

Following are situations which need system calls in OS:

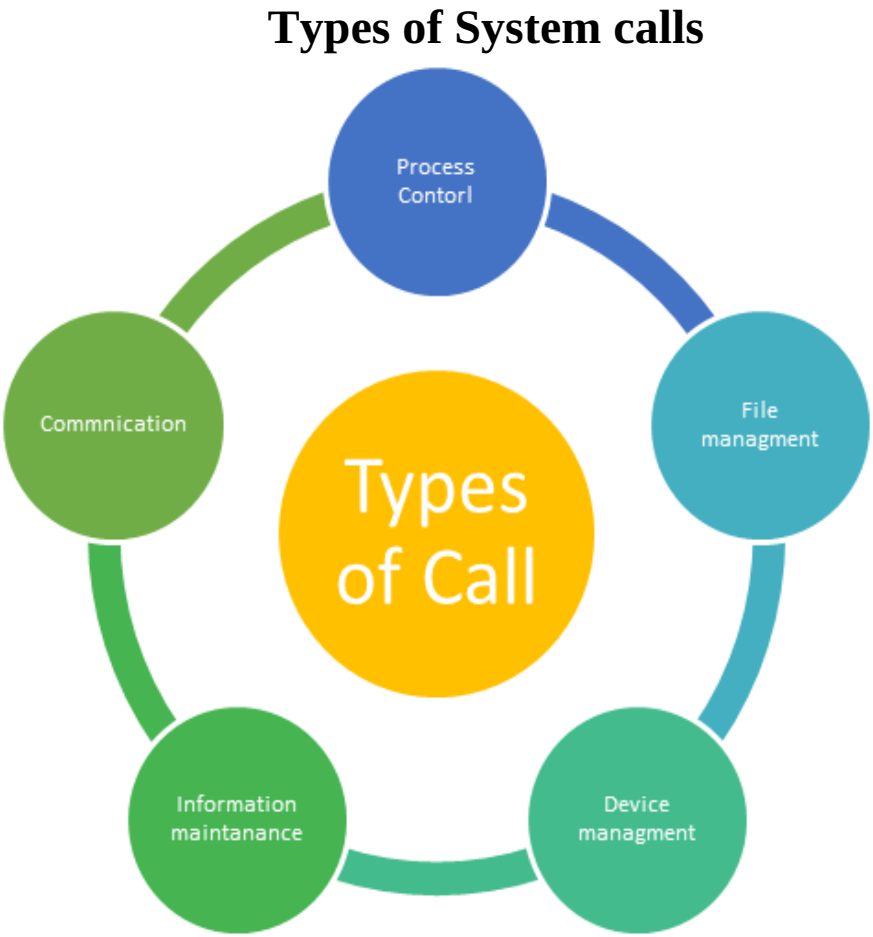
- Reading and writing from files demand system calls.
- If a file system wants to create or delete files, system calls are required.
- System calls are used for the creation and management of new processes.
- Network connections need system calls for sending and receiving packets.
- Access to hardware devices like scanner, printer, need a system call.

Types of System calls

Here are the five types of system calls used in OS:

- Process Control

- File Management
- Device Management
- Information Maintenance
- Communications



Process Control

This system calls perform the task of process creation, process termination, etc.

Functions:

- End and Abort
- Load and Execute
- Create Process and Terminate Process
- Wait and Signed Event
- Allocate and free memory

File Management

File management system calls handle file manipulation jobs like creating a file, reading, and writing, etc.

Functions:

- Create a file
- Delete file
- Open and close file
- Read, write, and reposition
- Get and set file attributes

Device Management

Device management does the job of device manipulation like reading from device buffers, writing into device buffers, etc.

Functions

- Request and release device
- Logically attach/ detach devices
- Get and Set device attributes

Information Maintenance

It handles information and its transfer between the OS and the user program.

Functions:

- Get or set time and date
- Get process and device attributes

Communication:

These types of system calls are specially used for interprocess communications.

Functions:

- Create, delete communications connections
- Send, receive message
- Help OS to transfer status information
- Attach or detach remote devices

Important System Calls Used in OS

- **wait()**
- **fork()**
- **exec()**
- **kill():**
- **exit():**

wait()

In some systems, a process needs to wait for another process to complete its execution. This type of situation occurs when a parent process creates a child process, and the execution of the parent process remains suspended until its child process executes.

The suspension of the parent process automatically occurs with a wait() system call. When the child process ends execution, the control moves back to the parent process.

fork()

Processes use this system call to create processes that are a copy of themselves. With the help of this system Call parent process creates a child process, and the execution of the parent process will be suspended till the child process executes.

exec()

This system call runs when an executable file in the context of an already running process that replaces the older executable file. However, the original process identifier remains as a new process is not built, but stack, data, head, data, etc. are replaced by the new process.

kill():

The kill() system call is used by OS to send a termination signal to a process that urges the process to exit. However, a kill system call does not necessarily mean killing the process and can have various meanings.

exit():

The exit() system call is used to terminate program execution. Specially in the multi-threaded environment, this call defines that the thread execution is complete. The OS reclaims resources that were used by the process after the use of exit() system call.